
PROJET ROVER

Commande en boucle ouverte du rover

AERO 2 – 2025/2026



-
- Objectif :**
- Comprendre le fonctionnement des encodeurs et moteurs à courant continu du rover
 - Analyser du code fourni
 - Faire les premiers mouvements – Commande en boucle ouverte du rover
-

1. Entrées-sorties du Rover

Signaux de sortie (signaux des capteurs) : position angulaire et vitesse angulaire de chaque roue

Par le biais de la carte installée au-dessous du Rover il est possible de récupérer les positions angulaires des roues et commander les moteurs correspondants. Chaque roue est reliée à un **motoréducteur avec encodeur** (d'où la présence de 6 fils entre les moteurs et la carte : canal A, canal B, 2 fils d'alimentation de l'encodeur, et deux fils d'alimentation du moteur).

L'encodeur délivre un train d'impulsions dont le nombre permet de déduire la valeur du déplacement ainsi que la vitesse de chaque roue. Si le **MOTEUR** fait un tour, N impulsions sont comptées. La présence de deux canaux A et B permet d'augmenter la précision par 4. La présence d'un réducteur permet aussi d'augmenter la précision sur l'angle de la roue. Si le facteur de réduction est 1/R, pour **chaque tour d'une ROUE** on compte **N*4*R impulsions**. La précision en degré sera donc $360/(4NR)$ degré par impulsion comptée.

Pour mieux comprendre le fonctionnement, se documenter sur l'Encodeur incrémental. (Utiliser Internet)

A partir du nombre d'impulsions comptées dans un intervalle de temps, on peut déduire la vitesse de la roue.

Signaux d'entrée (signaux des actionneurs) : entrée pwm ou tension d'alimentation de chaque moteur

Comme indiqué dans les séances précédentes, le moteur à courant continue fonctionne à une tension constante, mais il admet aussi une tension PWM. La tension moyenne à l'entrée sera donc la valeur du rapport cyclique (en %) multiplié par la tension d'alimentation (ici environ 7.5 volts).

2. Création d'un répertoire et configuration de l'environnement

Avant de commencer le codage sous VSCode, assurez-vous de la bonne configuration de **l'environnement de travail**.

- Dans le répertoire « Rover », créer un sous-répertoire de travail « Mouvements ».
- Dans VSCode, cliquer sur *Explorer* et ouvrir le répertoire (*folder*) « Mouvements ».
- Cliquer sur la barre de recherche en haut de l'écran puis sur « *Show and Run Commands* ».
- Taper « *MicroPico : Initialize MicroPico Project* » pour configurer l'environnement de travail avec MicroPython. Des fichiers « .micropico » et « .vscode » sont ajoutés dans votre répertoire de travail. Ils contiennent les informations de configuration, quelques réglages locaux de VSCode pour ce projet (police, couleurs, taille des lignes), etc.... Ne pas les supprimer.

3. Test du bon fonctionnement de la carte Raspberry Pi Pico W

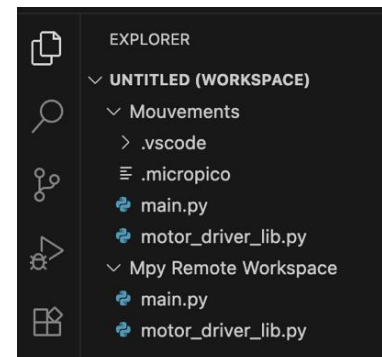
Pour tester la carte Pico W, vous pouvez tenter d'exécuter votre dernier script Python de clignotement de la LED interne à la carte dans une boucle infinie.

- Copier dans le répertoire votre dernier programme « main.py » de clignotement de la led LED.
- **Firmware (fichier en .uf2)** : Si ce n'est pas encore fait, charger le *firmware* et copier le dans la carte comme indiqué sur le site *micropython.org*. Ceci permet de lancer des scripts MicroPython dans la carte Pico W. Si c'est déjà fait dans le premier TP, ignorer cette étape.
- Exécuter le code sur Raspberry Pi Pico W :
Connecter le microcontrôleur à l'ordinateur, le message « *Connection to MicroPython board established* » s'affiche dans le coin en bas à droite de VSCode.
Deux façons sont possibles pour exécuter le script Python dans le microcontrôleur :
 1. On peut exécuter le code en cliquant sur « *Run* ». La LED devrait clignoter. Dans ce cas, la connexion avec le PC doit être maintenue et le code n'est pas importé sur le microcontrôleur.
 2. Pour une exécution de façon autonome, cliquer sur « *Toggle Mpy FS* » pour afficher, dans l'Explorer, les fichiers importés dans Pico W. Ces fichiers apparaîtront dans l'onglet « *Mpy Remote Workspace* ». Faire un clic droit sur le fichier « main.py » du répertoire de travail, puis sélectionner « *Upload file to Pico* ».
Débrancher le rover de votre ordinateur, puis allumer la batterie. La LED devrait clignoter.

4. Utilisation de la commande PWM pour déplacer le Rover

Nous allons utiliser un code existant pour se familiariser avec la commande PWM du moteur et les roues Mecanum. Notez que cette fois, on ne pourra pas lancer un test de mouvement des roues en liaison filaire car l'ordinateur ne peut pas fournir la puissance requise, qui provient des batteries.

- Dans Moodle du Projet ROVER, télécharger les fichiers « main.py » et « motor_driver_lib.py » dans l'onglet de la séance. Déplacer ces deux codes dans le nouveau répertoire de travail « Mouvements ».
- Dans VSCode, cliquer sur « *Toggle Mpy FS* » pour afficher les fichiers déjà existant dans le Pico W.
- Téléverser les deux codes sur le Pico W puis débranchez votre rover et posez-le au sol. Allumez la batterie. *Ne le laissez pas filer !*
- Pour gagner du temps, vous pouvez aussi garder le câble connecté au PC, allumer la batterie, cliquer sur « main.py » de votre répertoire de travail, et lancer « Run ». *Bien sûr, ne le laissez pas filer !*



Regardons de plus près le code « main.py » que nous venons d'exécuter sur le rover. Le module « machine » est à nouveau appelé, mais cette fois-ci la classe « I2C » est appelée en plus de la classe « Pin ». La classe « I2C » est utilisée pour configurer et gérer la **communication** avec des périphériques via le protocole I2C (Inter-Integrated Circuit). Un **protocole** est un ensemble de règles et de conventions définissant comment les données sont échangées entre des entités. Cette classe nous permet de faire communiquer le microcontrôleur et le driver des moteurs.

Un autre module, « motor_driver_lib », est importé sous l'alias « motor ». Ce module n'est pas intégré à MicroPython, mais contenu dans le fichier « motor_driver_lib.py ». Il faut savoir que ce module est fourni par le constructeur et que nous l'avons adapté au Rover (en lui indiquant le type de moteur que l'on utilise, la taille des roues, etc...). C'est dans ce module qu'est défini les fonctions qui nous intéressent. On va essayer de comprendre le contenu de ce fichier.

5. Analyse du code fourni

def set_motor_parameters(): pour configurer contrôleur des moteurs installé sous le Rover. Analyser cette fonction.

def read_total_encoder_counts(): retourne une liste de 4 éléments. Chaque élément représente la position angulaire de la roue, plus précisément le nombre d'impulsions (*nombre de ticks*) comptées par l'encodeur.

Remarquer la définition de la constante : **TICKS_PER_REVOLUTION = 2340** au début du fichier « main.py ».
Pourquoi cette valeur ?

def read_encoder_deltas(): compte le nombre d'impulsions des 4 roues sur un intervalle de temps de 10ms.

def calculate_rpm(ticks, TICKS_PER_REVOLUTION, SAMPLE_INTERVAL): calcule la vitesse rpm d'une roue. *ticks* est le nombre d'impulsions sur un intervalle de 10 ms.

Compléter cette fonction pour retourner le nombre de tours par minute (rpm) de la roue.

def control_motor_pwm(p1, p2, p3, p4): permet de commander par PWM des 4 moteurs.

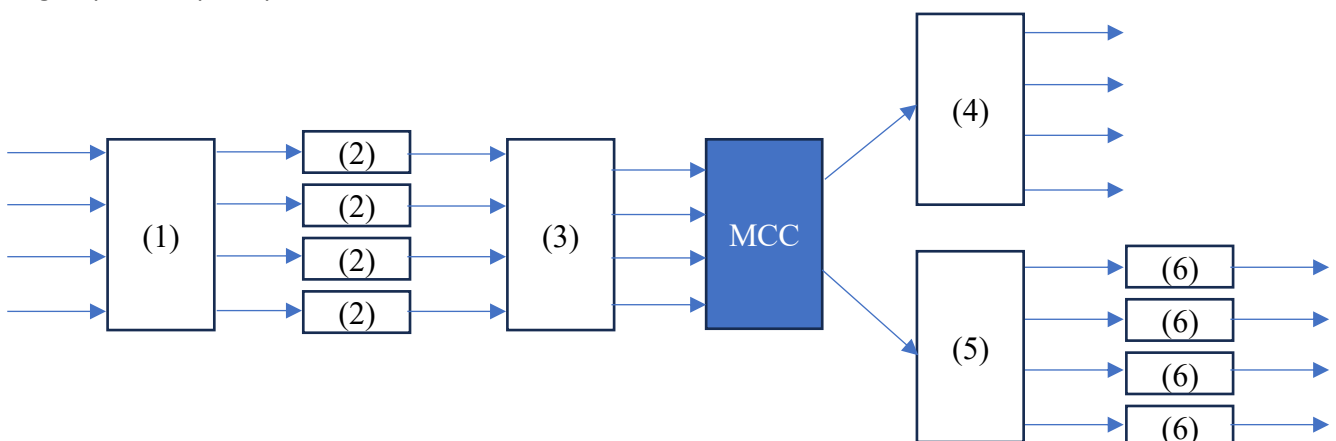
Les quatre valeurs p1 à p4 permettent de modifier le rapport cyclique du signal PWM envoyé à chaque moteur indépendamment. Les tensions moyennes d'alimentation (en volt) des moteurs sont donc le produit des rapports cycliques (en %) et de la tension maximale (environ 7,5 V).

La valeur par défaut est fixée à 1000. En variant les rapports cycliques, la tension moyenne varie, donc la vitesse de chaque roue varie. Cependant, si le rapport cyclique est trop faible, le couple moteur est insuffisant pour surmonter les frottements. En revanche, s'il est trop élevé, l'alimentation ne peut pas fournir assez de puissance pour supporter la charge du moteur, ce qui entraîne un arrêt.

def control_motor_voltage(v1, v2, v3, v4): commande les 4 moteurs en spécifiant la tension moyenne sur chaque moteur (en volts).

def voltage_to_pwm(voltage): convertit une valeur en volts en une valeurs PWM. Sachant que : 0 V -> PWM=0, et 7.34 V -> PWM=2000. **Compléter cette fonction (une simple règle de trois)**

Regroupons les principales fonctions dans le schéma suivant :



MCC représente les quatre moteurs à courant continu du Rover. Les flèches représentent les signaux d'entrées/sorties de ces différentes fonctions. Les carrés blancs représentent les différentes fonctions.

Que représentent les fonctions (1), (2), (3), (4), (5), (6) ?

Compléter le schéma en indiquant les différents signaux.

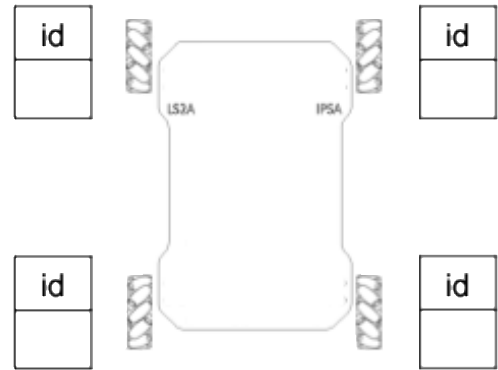
6. Mouvements du Rover, commande en boucle ouverte

Pour contrôler le moteur on utilise désormais la fonction : « control_motor_pwm » du module sous alias « motor » qui s'appelle sous la forme :

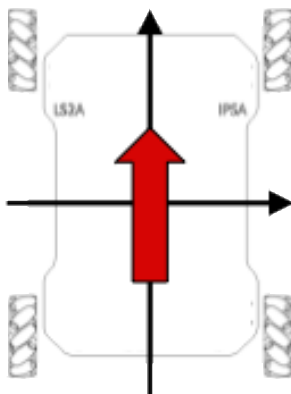
`motor.control_motor_pwm(p1, p2, p3, p4)`

On souhaite effectuer divers mouvements avec le rover en utilisant la commande PWM. Pour cela, il est nécessaire d'associer chaque roue à son attribut dans la fonction de contrôle du moteur.

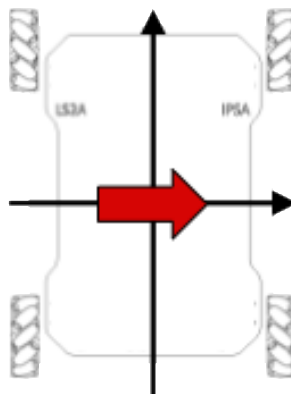
- Déterminer à quelle roue correspond chaque attribut p1 à p4 de la fonction de contrôle par PWM. Identifier les roues sur le schéma ci-contre. Faire des tests sur le Rover en mettant la valeur 1000 ou 0 aux p1 à p4.



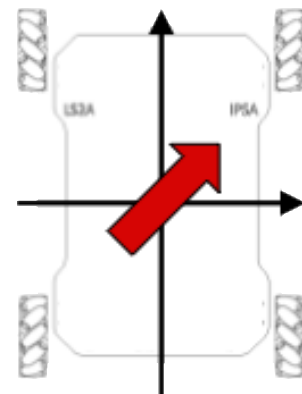
- Que ce passet-il si p1 est une valeur négative ? p1 = -1000 par exemple.
- Pour chaque moteur, faire des tests avec plusieurs PWM. Déterminer la valeur minimale qui permet de faire tourner les moteurs.
- Pour cette étape on se limite à aux valeurs -1000, 0 ou 1000 pour p1 à p4. Déterminez et notez les combinaisons d'attribut (p1,p2,p3,p4) qui permettent de réaliser les déplacements ci-dessous.



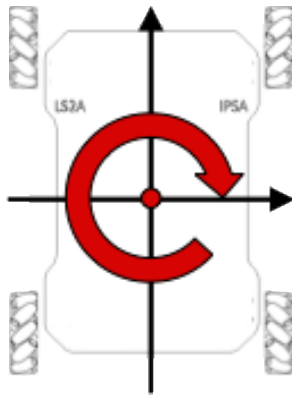
(__ , __ , __ , __)



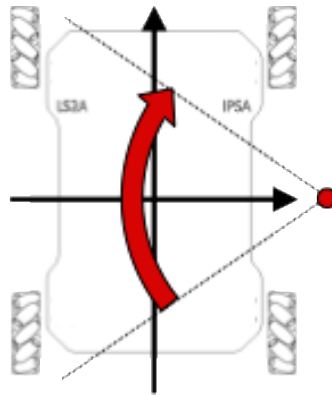
(__ , __ , __ , __)



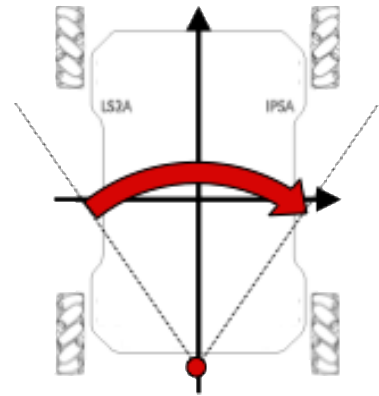
(__ , __ , __ , __)



(__ , __ , __ , __)

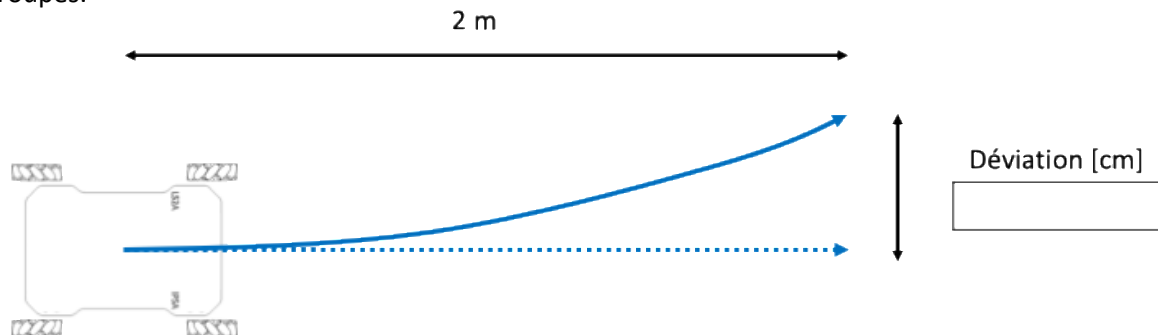


(__ , __ , __ , __)



(__ , __ , __ , __)

- Tester et valider ces mouvements sur le Rover.
- Vous connaissez le rayon des roues. Calculer le nombre de tours qu'il faut pour les roues pour faire un déplacement de 1 m. En déduire le nombre d'impulsions (nombre de ticks) qu'il faut.
- Modifier le code de main.py pour permettre à votre rover d'avancer en ligne droite sur une distance d'un mètre, puis de s'arrêter. A-t-il fait la distance prédite ? Analyser.
- Il est probable que votre rover ne progresse pas parfaitement en ligne droite et dévie légèrement de sa trajectoire. Mesurez cette déviation après avoir parcouru un mètre et comparez-la avec celle d'autres groupes.



En conclusion, à cause de cette commande en boucle ouverte, si la trajectoire du rover ne correspond pas à celle que l'on a donné en consigne, il serait nécessaire d'asservir le système, c'est-à-dire d'ajuster la consigne en s'appuyant sur des mesures (capteurs) pour améliorer le contrôle de la trajectoire.

- Faire en sorte que le Rover parcourt une distance de 1 m, s'arrête pendant 1 seconde, puis revient à sa position de départ en inversant les PWM. Est-il revenu exactement à sa position de départ ?
- Pour s'amuser encore plus, réaliser d'autres mouvements plus complexes avec le Rover (trajectoire carrée, circulaire...), tout en essayant de revenir à la position de départ. Essayer d'améliorer les performances tout en gardant la commande en boucle ouverte.